Operators Used in RStudio

Sushank Pokhrel (Homepage) (https://sushank.com.np)

Operators in R are symbols or functions that perform specific tasks on data, ranging from arithmetic to logical comparisons, data manipulation, and more. This section explains the different types of operators, their usage, and examples to help you understand their functionality.

1. Basic Arithmetic Operators

These operators are used for performing mathematical operations

- Addition (+)
- Subtraction (-)
- Multiplication (*)
- Division (/)
- Exponentiation (^ or **)
- Modulus (remainder of division) (%%)
- Integer division (quotient without remainder) (%/%)

```
2 + 2      # Addition
```

```
## [1] 4
```

```
10 - 3     # Subtraction
```

```
## [1] 7
```

```
4 * 5      # Multiplication
```

```
## [1] 20
```

```
20 / 4     # Division
```

```
## [1] 5
```

```
2^3        # Exponentiation
```

```
## [1] 8
```

```
sqrt(16)   # Square root
```

```
## [1] 4
```

2. Assignment Operators

Assignment operators in R are used to assign values to variables. These operators are essential for storing data so that it can be referenced and manipulated later in the code. R provides a few ways to assign values to variables, with the most commonly used ones being <-, ->, and =.

A. <- (Leftward Assignment)

This is the most commonly used and recommended assignment operator in R. It assigns the value on the right to the variable on the left. The use of <- is the R convention and preferred style, especially in the R community.

B. -> (Rightward Assignment)

This operator is rarely used and is the reverse of <-. It assigns the value on the left to the variable on the right. It is not commonly seen in R code but is valid for rightward assignment.

C. = (Equal Assignment)

The = operator can also be used for assignment, but it has some limitations in certain contexts (such as within function calls). Although it can work as an assignment operator, it's generally less preferred than <- because it may cause confusion in more complex expressions (e.g., in function arguments).

Example

```
# Assignment Operators
x <- 5         # Leftward assignment (preferred)
5 -> x         # Rightward assignment
x = 5          # Equal assignment (less preferred in some cases)
print(x)       # Outputs: 5
```

```
## [1] 5
```

3. Relational Operators

These operators compare values and return logical outputs (TRUE or FALSE):

Relational operators in R are used to compare values and return logical results—either TRUE or FALSE—based on the comparison. These operators are useful in conditional statements and loops where you want to compare numbers, strings, or other data types to perform specific actions.

A. Less than (<)

This operator checks if the value on the left is less than the value on the right. It returns TRUE if the left-hand side is less, and FALSE otherwise.

B. Less than or equal to (<=)

This operator checks if the value on the left is less than or equal to the value on the right. It returns TRUE if the left-hand side is less than or equal to the right-hand side, and FALSE otherwise.

C. Greater than (>)

This operator checks if the value on the left is greater than the value on the right. It returns TRUE if the left-hand side is greater, and FALSE otherwise.

D. Greater than or equal to (>=)

This operator checks if the value on the left is greater than or equal to the value on the right. It returns TRUE if the left-hand side is greater than or equal to the right-hand side, and FALSE otherwise.

E. Equal to (==)

This operator checks if the value on the left is equal to the value on the right. It returns TRUE if both values are equal, and FALSE otherwise.

F. Not equal to (!=)

This operator checks if the value on the left is not equal to the value on the right. It returns TRUE if the values are different, and FALSE if they are the same.

Examples:

```
# assigning variables to the values
x <- 10
y <- 20
```

```
x < y
```

```
## [1] TRUE
```

This checks if 10 is less than 20, and the result is TRUE because it is.

```
x <= y
```

```
## [1] TRUE
```

This checks if 10 is less than or equal to 20. Since 10 is indeed less, the result is TRUE.

```
x > y
```

```
## [1] FALSE
```

This checks if 10 is greater than 20. Since it is not, the result is FALSE.

```
x >= y
```

```
## [1] FALSE
```

This checks if 10 is greater than or equal to 20. Since it is neither greater nor equal, the result is FALSE.

```
x == y
```

```
## [1] FALSE
```

This checks if 10 is equal to 20. Since the two values are not the same, the result is FALSE.

```
x != y
```

```
## [1] TRUE
```

This checks if 10 is not equal to 20. Since they are different, the result is TRUE.

4. Sequence and Indexing

This section covers how to create sequences of numbers, control the spacing of those sequences, and how to access elements in vectors, lists, and data frames using indexing techniques in R. Use seq() for more control over the sequence. Access elements in vectors, lists, or data frames using [], [[]], or $.

A. Sequence Generation

You can create sequences in R quickly using the colon operator (:) or the seq() function.

Using the Colon Operator (:)

The colon operator is a shorthand for generating sequences of numbers with a default step of 1.

```
1:10
```

```
##  [1]  1  2  3  4  5  6  7  8  9 10
```

The colon operator is quick, but it's limited in that the step size is always 1. It is commonly used when you need a simple, incremental sequence.

Using the seq() Function

The seq() function provides more flexibility, allowing you to specify a step size (by) or the number of values (length.out) in the sequence.

```
seq(1, 10, by = 2)
```

```
## [1] 1 3 5 7 9
```

Here, by = 2 indicates that each subsequent number in the sequence increases by 2. Therefore, R generated a sequence from 1 to 10, with a step of 2.

Using seq() with length.out

You can also specify the number of values you want in the sequence, and R will automatically adjust the step size to fit.

```
seq(1, 5, length.out = 4)
```

```
## [1] 1.000000 2.333333 3.666667 5.000000
```

This sequence has 4 equally spaced values from 1 to 5, with R calculating the appropriate step size.

B. Indexing

Indexing allows you to access specific elements from vectors, lists, and data frames.

Indexing a Vector

Vectors in R are indexed using square brackets ([]). Indexing in R starts at 1, meaning the first element of a vector is indexed as 1.

```
vector <- c("a", "b", "c", "d")
print(vector[2])
```

```
## [1] "b"
```

Here, vector[2] returns the 2nd element of the vector, which is “b”.

Indexing a List

Lists are indexed using [[ ]] to access individual elements by position, or $ to access named elements.

```
list_example <- list(a = 1, b = c(2, 3), c = "hello")


print(list_example[[2]])
```

```
## [1] 2 3
```

In this example, list_example[[2]] returns the 2nd element of the list, which is the vector c(2, 3).

```
print(list_example[[1]])
```

```
## [1] 1
```

Here, list_example[[1]] accesses the first element in the list, which is 1.

```
print(list_example$c)
```

```
## [1] "hello"
```

Here, list_example$c accesses the element with the name “c”, returning “hello”. This method is useful when you want to access an element by its name in a list.

5. Logical Operators

These operators are commonly applied to boolean data types (TRUE or FALSE) to evaluate conditions or expressions.They are used to evaluate conditions Here's an explanation of each operator and how they are used:

A. Element-wise AND (&)

The element-wise AND operator (&) performs a logical AND operation between corresponding elements in two boolean vectors (or arrays), returning TRUE if both elements are TRUE, and FALSE otherwise.

This operator is used when you need to compare the individual elements of two logical vectors, element by element.

Example:

```
TRUE & FALSE
```

```
## [1] FALSE
```

In this case, the result is FALSE because both sides are not TRUE (one is TRUE, and the other is FALSE).

```
c(TRUE, FALSE, TRUE) & c(FALSE, FALSE, TRUE)
```

```
## [1] FALSE FALSE  TRUE
```

This compares each element from the two vectors:

- TRUE & FALSE → FALSE
- FALSE & FALSE → FALSE
- TRUE & TRUE → TRUE

B. Element-wise OR (|)

The element-wise OR operator (|) performs a logical OR operation between corresponding elements in two boolean vectors, returning TRUE if at least one element is TRUE, and FALSE if both elements are FALSE.

Similar to &, but checks if at least one side is TRUE in each element-wise comparison.

Example:

```
TRUE | FALSE
```

```
## [1] TRUE
```

In this case, the result is TRUE because one side is TRUE.

```
c(TRUE, FALSE, TRUE) | c(FALSE, FALSE, TRUE)
```

```
## [1]  TRUE FALSE  TRUE
```

This compares each element from the two vectors:

- TRUE | FALSE → TRUE
- FALSE | FALSE → FALSE
- TRUE | TRUE → TRUE

C. Short-circuit AND (&&)

The short-circuit AND operator (&&) is used to evaluate logical expressions where the second condition is not evaluated if the first condition is FALSE. This is because, in an AND operation, if the first condition is FALSE, the result will always be FALSE regardless of the second condition.

Typically used for conditions where you only need to check the first condition in a logical expression, saving computation time.

Example:

```
FALSE && TRUE
```

```
## [1] FALSE
```

In this case, the result is FALSE because the first condition (FALSE) ensures that the entire expression is FALSE without checking the second condition.

- In &&, the evaluation stops as soon as the result is determined (short-circuiting). If the first condition is FALSE, the second condition isn't evaluated at all.

D. Short-circuit OR (||)

The short-circuit OR operator (||) works similarly to &&, but for the OR operation. If the first condition is TRUE, the second condition is not evaluated because, in an OR operation, if the first condition is TRUE, the result will always be TRUE.

This is used in conditions where you want to evaluate only the first condition to save computation if possible.

Example:

```
TRUE || FALSE
```

```
## [1] TRUE
```

In this case, the result is TRUE because the first condition (TRUE) makes the whole expression TRUE, so the second condition isn't evaluated.

- Like &&, the || operator short-circuits and stops evaluating as soon as the result is known.

E. NOT (!)

The NOT operator (!) inverts the boolean value of the operand. If the operand is TRUE, it returns FALSE, and if the operand is FALSE, it returns TRUE.

This is used to negate or reverse a logical condition.

Example:

```
!TRUE
```

```
## [1] FALSE
```

Here, the ! operator negates the TRUE value and returns FALSE.

```
!FALSE
```

```
## [1] TRUE
```

Here, the ! operator negates FALSE and returns TRUE.

6. Matrix Operations

In R, matrices are two-dimensional data structures that allow you to store and manipulate data in rows and columns. R provides a wide range of functions for matrix operations, such as addition, multiplication, transposition, and more. These operations are essential for linear algebra, statistical modeling, and various numerical computations.

A. Matrix Multiplication (%*%)

Matrix multiplication is done using the %*% operator. It is different from element-wise multiplication.

In matrix multiplication, the number of columns in the first matrix must match the number of rows in the second matrix. The result will have the dimensions of the first matrix's rows and the second matrix's columns.

Example:

```
A <- matrix(1:4, nrow = 2)  # Create a 2x2 matrix A
print(A)
```

```
##      [,1] [,2]
## [1,]    1    3
## [2,]    2    4
```

```
B <- matrix(5:8, nrow = 2)  # Create a 2x2 matrix B
print(B)
```

```
##      [,1] [,2]
## [1,]    5    7
## [2,]    6    8
```

Now, Performing the matrix multiplication.

```
result <- A %*% B  # Matrix multiplication using %*%

# Print the result
print(result)
```

```
##      [,1] [,2]
## [1,]   23   31
## [2,]   34   46
```

B. Element-wise Operations (*, +, -, etc.):

To perform element-wise multiplication, addition, or subtraction, use the *, +, and - operators, respectively.

Example

```
A <- matrix(1:4, nrow = 2)  # Create a 2x2 matrix A
print(A)
```

```
##      [,1] [,2]
## [1,]    1    3
## [2,]    2    4
```

```
B <- matrix(5:8, nrow = 2)  # Create a 2x2 matrix B
print(B)
```

```
##      [,1] [,2]
## [1,]    5    7
## [2,]    6    8
```

```
sum_matrix <- A + B

print(sum_matrix)
```

```
##      [,1] [,2]
## [1,]    6   10
## [2,]    8   12
```

Similarly, Other operations can be done.

C. Matrix Transposition:

The transpose of a matrix is obtained using the t() function in R. It flips the matrix over its diagonal.

Example:

```
A <- matrix(1:4, nrow = 2)  # Create a 2x2 matrix A
print(A)
```

```
##      [,1] [,2]
## [1,]    1    3
## [2,]    2    4
```

```
A_transposed <- t(A)
print(A_transposed)
```

```
##      [,1] [,2]
## [1,]    1    2
## [2,]    3    4
```

D. Inverse of a Matrix

To find the inverse of a matrix, use the solve() function, provided the matrix is square and non-singular (its determinant is not zero).

```
A <- matrix(1:4, nrow = 2)  # Create a 2x2 matrix A
print(A)
```

```
##      [,1] [,2]
## [1,]    1    3
## [2,]    2    4
```

```
A_inv <- solve(A)  # Finds the inverse of matrix A
print(A_inv)
```

```
##      [,1] [,2]
## [1,]   -2  1.5
## [2,]    1 -0.5
```

7. Special Operators

Integer Division (%/%): Returns the integer quotient. Modulus (%%): Returns the remainder of a division.

Example:

```
10 %/% 3    # 3 (quotient)
```

```
## [1] 3
```

```
10 %% 3     # 1 (remainder)
```

```
## [1] 1
```

8. Formula Interface

The formula interface in R is commonly used in regression modeling and other statistical functions to represent relationships between variables. It provides a concise way to specify mathematical relationships between dependent and independent variables in a model. This syntax is particularly useful when working with functions like lm() (linear model) and glm() (generalized linear model).

The formula interface uses the syntax response ~ predictor where: response is the dependent variable (the outcome). predictor is the independent variable (the predictor or feature). You can also include multiple predictors and interaction terms, like y ~ x1 + x2 or y ~ x1 * x2, where the * operator represents interaction between x1 and x2.

Example:

```
# Linear model using formula
model <- lm(y ~ x, data = data.frame(y = 1:10, x = 11:20))
summary(model)
```

```
## Warning in summary.lm(model): essentially perfect fit: summary may be
## unreliable
```

```
##
## Call:
## lm(formula = y ~ x, data = data.frame(y = 1:10, x = 11:20))
##
## Residuals:
##        Min         1Q     Median         3Q        Max
## -2.280e-15 -8.925e-16 -2.144e-16  4.221e-16  4.051e-15
##
## Coefficients:
##               Estimate Std. Error    t value Pr(>|t|)
## (Intercept) -1.000e+01  3.095e-15 -3.231e+15   <2e-16 ***
## x            1.000e+00  1.963e-16  5.093e+15   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.783e-15 on 8 degrees of freedom
## Multiple R-squared:      1,  Adjusted R-squared:      1
## F-statistic: 2.594e+31 on 1 and 8 DF,  p-value: < 2.2e-16
```

- lm() is the function for fitting linear models. In this case, we are using lm(y ~ x) to model y as a function of x.
- The formula y ~ x specifies that y is the dependent variable, and x is the independent variable.
- The data argument is a data frame containing the variables y and x.
- The summary(model) function prints a summary of the fitted model, including coefficients, residuals, and other statistical details.
- Output: The output will provide the coefficients for the intercept and slope, R-squared value, and other statistical measures for evaluating the model.

9. Piping in R

Piping is a powerful tool in R that allows you to chain together multiple operations in a clear, readable manner. There are two types of pipes commonly used in R: the native pipe (|>) introduced in R 4.1.0 and the Magrittr pipe (%>%), which is popularized by the dplyr package.

A. Native Pipe (|>)

The native pipe was introduced in R 4.1.0 and provides a more efficient way to chain functions. The operator |> passes the result of the left-hand side expression to the first argument of the function on the right-hand side.

Example

```
numbers <- 1:10
numbers |> mean() |> round(2)  # Chain operations
```

```
## [1] 5.5
```

- numbers is a vector from 1 to 10.
- numbers |> mean() calculates the mean of the vector.
- |> round(2) rounds the result to 2 decimal places.
- The result of the entire operation will be the rounded mean of the numbers.
- The output would be the rounded mean value of the numbers (which is 5.5 in this case).

B. Magrittr Pipe (%>%):

The Magrittr pipe (%>%) is widely used in the dplyr and tidyverse packages. It is very similar to the native pipe but allows for more flexibility, especially when working with complex data manipulation tasks in data frames. It allows you to pass the result of one function directly to the next function in the chain without needing to explicitly reference the intermediate results. Below is an example. You need to install and load dplyr or magrittr or tidyverse package to run this pipe operation.

Example

```
library(dplyr)
```

```
## Warning: package 'dplyr' was built under R version 4.3.3
```

```
##
## Attaching package: 'dplyr'
```

```
## The following objects are masked from 'package:stats':
##
##     filter, lag
```

```
## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union
```

```
data <- data.frame(x = 1:10, y = 11:20)
result <- data %>%
  mutate(z = x + y) %>%
  filter(z > 25)
print(result)
```

```
##    x  y  z
## 1  8 18 26
## 2  9 19 28
## 3 10 20 30
```

- data is a data frame with two columns x and y.
- %>% is used to chain operations together.
- The first operation creates a new column z which is the sum of x and y using mutate(z = x + y).
- The second operation filters rows where z is greater than 25 using filter(z > 25).
- The resulting data frame result will contain only rows where the sum of x and y is greater than 25.